

Sisteme de Operare. Laborator 2

Honorius Gâlmeanu
galmeanu@unitbv.ro

February 26, 2023

1 Editoare de text

Editoarele de text sunt aplicații interactive folosite pentru editarea programelor, a fișierelor de configurare etc. Multe vin cu capabilități precum syntax highlighting, autocomplete, navigare, căutare, înlocuire și altele. Enumerăm câteva dintre ele:

- **vim** este un editor în mode text disponibil pe orice versiune de Linux; suportă comenzi automate, împărțirea ecranului în sesiuni, scheme de culoare. Este destul de dificil de învățat, datorită interacțiunii în mod text, dar este foarte popular în comunitatea Linux;
- **nano** este o revizie a unui editor mai vechi denumit Pico; este un editor minimal, tot în mod text, mult mai ușor de învățat decât vim; lista de comenzi este totuși destul de scurtă;
- **gedit** este un editor care vine cu GNOME Desktop Environment. Este un editor grafic, intuitiv, adaptat pentru majoritatea limbajelor de programare;
- **geany** este tot un editor grafic, destul de bine adaptat majorității limbajelor, și are o amprentă de resurse mai redusă decât precedentul;
- **Visual Studio Code** este un editor cross-platform; deși creat de Microsoft, funcționează pe Windows, Linux sau Mac. Se integrează bine cu compilatorul și depanatorul GNU, dar consumă destul de multe resurse.



Figure 1: Editorul de text nano și comenzile sale.

În cele ce urmează vom folosi editorul nano (figura 1).

2 Redirectarea intrării și ieșirii

Ieșirea unui program se redirectează cu `>`. Comanda *echo* reproduce la ieșire ceea ce este specificat în linia de comandă. Comanda *cat* tipărește fișierul sau fișierele specificate în linia de comandă. În acest context, iată cum ar arăta redirectarea ieșirii:

Command Line

```
$ echo ceva > file.txt
$ cat file.txt
ceva
```

Programul *gzip* comprimă un fișier dat ca parametru. Dacă acesta lipsește, se citește intrarea. Tipărirea unui fișier comprimat se face cu **zless** (interactiv) sau **zcat**:

Command Line

```
$ cat file.txt | gzip > file.txt.gz
$ zcat file.txt.gz
ceva
```

Intrarea se redirectează cu `<`. Comanda *cat* tipărește fie fișierul dat ca parametru fie intrarea, dacă acesta lipsește. Astfel, următoarele două comenzi produc același efect:

Command Line

```
[user@fedora ~]$ cat < file.txt
ceva
[user@fedora ~]$ cat file.txt
ceva
```

Adăugarea la fișierul existent se face cu `>>`. Adăugarea se face fără a introduce terminatorul de linie (Line Feed, LF). Acesta este introdus de comanda *echo*. Dacă vrem ca *echo* să nu introducă LF (new line), folosim opțiunea `-n`:

Command Line

```
[user@fedora ~]$ echo "linia 2" >> file.txt
[user@fedora ~]$ cat file.txt
ceva
linia 2
[user@fedora ~]$ echo -n "linia 3" >> file.txt
[user@fedora ~]$ echo " continuare" >> file.txt
[user@fedora ~]$ echo "linia 4" >> file.txt
[user@fedora ~]$ cat file.txt
ceva
linia 2
linia 3 continuare
linia 4
```

Secvența `<<` are alt rol; ea se numește **here-document**, și indică secvența de stop în procesarea intrării. De exemplu, citirea continuă în cazul următor până introducem secvența declarată de here-document:

Command Line

```
[user@fedora ~]$ echo << EOF
> ala bala
> portocala
> EOF

[user@fedora ~]$
```

3 Programe de backup

Arhivarea fișierelor se face folosind comanda *tar*. Iată cum se arhivează folder-ul *extra*:

Command Line

```
[user@fedora ~]$ tar -cvf extra.tar extra/
extra/
extra/func.c
extra/main.c
extra/func.h
extra/Makefile
extra/Makefile2
extra/Makefile3
```

S-a folosit opțiunea *-c* ce înseamnă construcția arhivei, *-v* de la "verbose" care afișează tot ceea ce se face și *-f* care trebuie să fie neapărat urmată de numele arhivei care se creează. Mai departe, linia de comandă conține toate fișierele și directoarele incluse în arhivă.

Operația de dezarhivare se face cu opțiunea *-x* (opusul lui *-c*) ce indică extragerea informațiilor, iar opțional putem folosi *-C* pentru a indica folder-ul destinație:

Command Line

```
[user@fedora ~]$ tar -xf extra.tar -C dup
[user@fedora ~]$ ls -lR dup/
dup/:
total 0
drwxr-xr-x. 2 user user 98 Feb 25 15:02 extra

dup/extra:
total 24
-rw-r--r--. 1 user user 102 Feb 25 10:44 func.c
-rw-r--r--. 1 user user 25 Feb 25 10:44 func.h
-rw-r--r--. 1 user user 373 Feb 25 10:47 main.c
-rw-r--r--. 1 user user 85 Feb 25 11:04 Makefile
-rw-r--r--. 1 user user 495 Feb 25 11:09 Makefile2
-rw-r--r--. 1 user user 353 Feb 25 11:12 Makefile3
```

Pentru *ls* am folosit opțiunea *-l* pentru "long list", respectiv *-R* pentru parcurgerea recursivă a directorilor găsite.

De regulă arhivarea o cuplăm cu comprimarea folosind programele *gzip* și *gunzip*; aceasta se poate face fie prin piping (folosirea *|* pentru preluarea ieșirii generate de primul proces și scrierea la intrarea celui de-al doilea), fie specificând opțiunea *-z*. Următoarele comenzi sunt echivalente:

Command Line

```
[user@fedora ~]$ tar -c extra/ | gzip > extra.tgz
[user@fedora ~]$ gunzip -c extra.tgz | tar -x -C dup/

[user@fedora ~]$ tar -zcf extra.tar.gz extra/
[user@fedora ~]$ tar -zxvf extra.tar.gz -C dup/
```



Temă: În timp ce rulați aceste comenzi, informați-vă folosind comanda *man* despre opțiunile folosite.

Un alt utilitar pentru compresie este *zip*:

Command Line

```
[user@fedora ~]$ zip -r extra.zip extra/*
adding: extra/Makefile (deflated 24%)
adding: extra/Makefile2 (deflated 46%)
adding: extra/Makefile3 (deflated 37%)
adding: extra/func.c (deflated 18%)
adding: extra/func.h (deflated 8%)
adding: extra/main.c (deflated 44%)
[user@fedora ~]$ unzip extra.zip
Archive:  extra.zip
replace extra/Makefile? [y]es, [n]o, [A]ll, [N]one, [r]ename: A
inflating: extra/Makefile
inflating: extra/Makefile2
inflating: extra/Makefile3
inflating: extra/func.c
inflating: extra/func.h
inflating: extra/main.c
[user@fedora ~]$ ls -l
total 30
drwxr-xr-x. 3 user user   19 Feb 25 15:55 dup
drwxr-xr-x. 2 user user   98 Feb 25 16:13 extra
-rw-r--r--. 1 user user 10240 Feb 25 16:00 extra.tar
-rw-r--r--. 1 user user   940 Feb 25 16:00 extra.tar.gz
-rw-r--r--. 1 user user   940 Feb 25 16:04 extra.tgz
-rw-r--r--. 1 user user  1817 Feb 25 16:12 extra.zip
```



Temă: Ce părere aveți despre performanța acestor trei arhivatoare, *tar*, *gzip* și *zip*? (*tar* nu face compresie, doar concatenează).

4 SED: Stream Editor

Sed este un program de editare și transformare a textului; pe baza unui șablon dat, *sed* prelucrează un text, căutând anumite secvențe și realizând înlocuiri, în mod automat, din linie de comandă. Sintaxa sa este:

Command Line

```
$ sed [optiuni] comenzi [fisier]
```

Comanda pentru căutare și înlocuire este *s/regexp/replacement/*; se caută potriviri pentru expresia regulată *regexp* și se înlocuiește cu *replacement*. Considerăm următorul text:

Command Line

```
[user@fedora ~]$ cat file.txt
words keep on coming
on and on and ON here
and onto here also
```

Dorim să înlocuim toate aparițiile cuvântului "on" cu "off":

Command Line

```
[user@fedora ~]$ sed 's/on/off/' file.txt
words keep off coming
off and on and ON here
and offto here also
```

Se pare că am înlocuit mai mult decât era necesar; modificăm expresia regulată în așa fel încât să potrivim doar cuvintele, nu și subșirurile:

Command Line

```
[user@fedora ~]$ sed 's/on /off /' file.txt
words keep off coming
off and on and ON here
and onto here also
```

Înlocuirea nu prinde și o a doua potrivire pe aceeași linie, așa încât vom adăuga flag-ul *g* (global):

Command Line

```
[user@fedora ~]$ sed 's/on /off /g' file.txt
words keep off coming
off and off and ON here
and onto here also
```

Flag-ul *i* va face căutarea case-insensitive:

Command Line

```
[user@fedora ~]$ sed 's/on /off /gi' file.txt
words keep off coming
off and off and off here
and onto here also
```

Să presupunem că vrem să căutăm cuvântul "here" și doar să marcăm prezenta lui între paranteze; ca *replacement* vom pune între paranteze contextul potrivit, a cărui variabilă substituit este *&*:

Command Line

```
[user@fedora ~]$ sed 's/here/(&)/g' file.txt
words keep on coming
on and on and ON (here)
and onto (here) also
```

Dorim să inversăm primele două cuvinte din fiecare linie între ele. Pentru aceasta, folosim mai multe elemente:

- paranteza este folosită pentru a grupa termenii expresiei regulate, iar pentru a nu fi interpretată ca atare trebuie prefixată cu caracterul backslash \ ;
- un cuvânt este definit ca orice literă sau cifră urmat de oricâte alte litere sau cifre (chiar și niciuna);
- setul de care ține un un caracter este reprezentat cu paranteze pătrate;
- steluța * reprezintă zero sau mai multe apariții;
- pe partea de replacer, cele două contexte potrivite se reprezintă cu caracterul backslash \ urmat de numărul contextului.

Command Line

```
[user@fedora ~]$ sed 's/\([a-zA-Z0-9][a-zA-Z0-9]*\) \([a-zA-Z0-9][a-zA-Z0-9]*\)
/2 \1/' file.txt
keep words on coming
and on on and ON here
onto and here also
```

Expresia nu este suficient de completă, deoarece am dori ca un cuvânt să fie detectat ca orice secvență de caractere care nu conține spații; definiția mai precisă se obține definind setul cu operatorul "orice caracter în afară de", reprezentat de ~ (denumit în engleză "caret", putere):

Command Line

```
[user@fedora ~]$ sed 's/\([^ ]*\) \([^ ]*\)/2 \1/' file.txt
keep words on coming
and on on and ON here
onto and here also
```

În fine, ultimul truc este că *sed* poate să facă toate aceste lucruri in-place în fișier, trebuie doar să specificăm opțiunea *-i*:

Command Line

```
[user@fedora ~]$ sed -i 's/\([^ ]*\) \([^ ]*\)/2 \1/' file.txt
[user@fedora ~]$ cat file.txt
keep words on coming
and on on and ON here
onto and here also
```

5 Utilitarul grep. Sort și uniq

Grep este o unealtă care tipărește doar liniile care se potrivesc cu un pattern dat; foarte util pentru căutări rapide. Sintaxa este:

Command Line

```
$ grep [optiuni] pattern [fisier]
```

De exemplu, următoarea comandă afișează toate liniile de text, împreună cu numărul lor (opțiunea *-n*) unde apare cuvântul "grep" în pagina de manual a lui *grep*:

Command Line

```
$ man grep | grep -n grep
```

Dacă dorim să afișăm liniile care nu conțin cuvântul "grep" (toate celelalte linii), folosim opțiunea `-v`:

Command Line

```
$ man grep | grep -nv grep
```

Grep poate fi făcut case-insensitive cu opțiunea `-i`.

O problemă interesantă este găsirea locației unor anumite fișiere în filesystem. De exemplu, dorim să găsim unde se află fișierele header. Pentru aceasta vom folosi utilitarele *sort* care sortează liniile unui text dat la intrare, respectiv *uniq*, care suprimă liniile identice consecutive într-un text:

Command Line

```
[user@fedora ~]$ find / -name "*.h" | sed 's/\(.*\)\/.*/\1/' | sort | uniq
/home/user/extra
/usr/include
/usr/include/arpa
/usr/include/asm
/usr/include/asm-generic
...
```

Comanda *find* caută toate fișierele header, pornind din rădăcină. Se va genera o listă de căi care este preluată de *sed*; acesta va căuta pe fiecare linie poziția ultimului caracter slash / și va afișa doar ceea ce se află înaintea lui. Liniile astfel generate sunt sortate și se reține doar unul dintre duplicate.

6 Script-uri în bash

Fișierele script pot realiza prelucrări complexe prin înlănțuire de comenzi. De regulă ele au extensia `.sh` (dar pot avea orice extensie), iar dacă au setat dreptul de execuție pot fi apelabile direct, ca și celelalte comenzi.

Iată un exemplu de implementare a algoritmului lui Euclid într-un script bash:

Command Line

```
1: #!/bin/bash
2:
3: cmmdc () {
4:     a=$1
5:     b=$2
6:     echo "> $a $b"
7:
8:     if [ $b -eq 0 ]; then
9:         return $a
10:    fi
11:
12:    c=$(( $a % $b ))
13:    cmmdc $b $c
14:    return $?
15: }
16:
17: echo "parametrii sunt: $0, primul: $1, al doilea: $2"
18:
19: if [ $1 -lt $2 ]; then
20:     echo "ordine incorecta"
21:     exit 1
22: fi
23:
24: cmmdc1 $1 $2
25: echo "c.m.m.d.c. este: $?"
```

Implementarea folosește o funcție recursivă. Reținem următoarele aspecte:

- prima linie va conține, imediat după semnul de comentariu # și semnul de exclamare, calea spre interpretorul de comenzi. Este necesar dacă facem fișierul executabil și-l lansăm în al doilea mod;
- funcția este definită între liniile 3 - 15. Variabilele se accesează prin prefixarea numelui lor cu semnul \$, iar \$1, \$2 ... este notația folosită pentru accesarea argumentelor funcției. Notația \$@ se referă la toți parametrii, vezi linia 17;
- linia 6 este folosită doar pentru depanare și poate fi comentată prin prefixarea cu # sau ștersă;
- în linia 8 apare o expresie logică, în care variabila \$b este comparată cu 0. Operatorii de egalitate nu se folosesc decât pentru șiruri, pentru numere avem `-eq` pentru egalitate, `-lt` pentru "mai mic decât", `-le` pentru "mai mic sau egal" etc.;
- în linia 12 avem calcularea unei expresii aritmetice. Acest lucru se face cu operatorul `$( )`, iar în interior putem folosi orice operație aritmetică; aici am folosit restul împărțirii;
- la linia 13 s-a apelat recursiv funcția; se observă că acest lucru se face exact în același mod în care se invocă script-ul original (nu folosim paranteze);
- linia 21 invocă apelul de sistem `exit` care oprește procesarea și întoarce un cod de eroare;
- pe liniile 14 și 25 vedem cum se inspectează valoarea întoarsă de funcție - folosind variabila specială `$?`.

Execuția se poate face în două moduri, după ce setăm dreptul de execuție pentru al doilea mod:

Command Line

```
[user@fedora ~]$ sh cmmdc.sh 273 26
parametrii sunt: 273 26, primul: 273, al doilea: 26
> 273 26
> 26 13
> 13 0
c.m.m.d.c. este: 13
[user@fedora ~]$ chmod +x cmmdc.sh
[user@fedora ~]$ ./cmmdc.sh 273 26
parametrii sunt: 273 26, primul: 273, al doilea: 26
> 273 26
> 26 13
> 13 0
c.m.m.d.c. este: 13
```

Funcția de calcul poate fi și iterativă. Observați cum s-a folosit negația în expresia logică a lui *while*:

Command Line

```
cmmdc () {
    a=$1
    b=$2

    while [ ! $b -eq 0 ]; do
        c=$(( $a % $b ))
        a=$b
        b=$c
    done

    return $a
}
```

7 Exerciții

Exercițiul 1

Executați tot laboratorul pas cu pas pe mașina voastră. Folosiți paginile de manual și explicați tema propusă.

Exercițiul 2

Scrieți un script bash care calculează termenul Fibonacci de rang n.

Exercițiul 3

Folosiți *find* și *grep* pentru a afla ce directoare conțin fișiere cu extensia *.conf*. Scrieți un script bash care calculează suma totală a dimensiunilor directoarelor ce conțin aceste fișiere (atenție să nu aveți același director de mai multe ori). Folosiți, de exemplu, construcția cu apostrof invers ' pentru a parcurge toate liniile dintr-un fișier:

```
[user@fedora ~]$ cat some.txt
one
two
three
[user@fedora ~]$ for i in `cat some.txt`; do echo "> $i" ; done
> one
> two
> three
```

Exercițiul 4

Compuneți un fișier script bash care scrie 100 de linii într-un fișier cu numele *file.txt*. Liniile vor fi de forma "linia n: continut", unde "n" este de fapt numărul liniei, începând cu 1.

Exercițiul 5

Pentru fișierul creat la întrebarea anterioară, folosiți *sed* ca să înlocuiți toate aparițiile cuvântului "linia" cu "line", iar "continut" cu "content". Păstrați neschimbate numerele de linie, dar ștergeți ":" și înlocuiți-l cu " - ". O linie de forma "linia 3: continut" se va schimba în "line 3 - content".

Bibliografie

1. J. Ellingwood, B. Hogan, "The Basics of Using the Sed Stream Editor to Manipulate Text in Linux", <https://www.digitalocean.com/community/tutorials/the-basics-of-using-the-sed-stream-editor-to-m>
2. L. Reynolds, "Bash Scripting Tutorial", <https://linuxconfig.org/bash-scripting-tutorial>
3. Machtelt Garrels, "Introduction to Linux - A Hands-on Guide", <https://tldp.org/LDP/intro-linux/intro-linux.pdf>